

Отчет по классу Power -- Сообщение о проблеме

Ошибка 1

При любом положительном значении b метод `pow` класса `Power` пакета `root.pow` возвращает 0 вместо a^b (13:20 13.02.26)

Контекст

2.1 До вызова были объявлены переменные `int a=2`, `int b=5`, создан объект `Power power` дефолтным конструктором

2.2 Был вызван метод `power.pow(a,b)` и результат сохранен в `int c`

2.3 Фактически возвращаемое значение: `c = 0`

2.4 Ожидаемое значение: `c = 32`

2.5 Нарушено требование 2: метод должен возвращать результат возведения первого аргумента в степень, равную второму

Исправление

Проблема в инициализации `r`: при `r = 0` дальнейшие умножения всегда дают 0. Нужно начать с 1.

```
public int pow(int a, int b) {  
    int r = 1;  
  
    while (b > 0) {  
        if ((b & 1) != 0) r *= a;  
        r *= r;  
        b >>= 1;  
    }  
  
    return r;  
}
```

Ошибка 2

При некоторых значениях аргументов a и b метод `pow` класса `Power` возвращает некорректное значение из-за неправильной схемы быстрого возведения в степень (13:35 13.02.26)

Контекст

2.1 До вызова: `int a=2`, `int b=6`, `Power power` создан

2.2 Воздействие: вызов `power.pow(a,b)`

2.3 Фактически возвращаемое значение после исправления из Ошибки 1: `c = 8`

2.4 Ожидаемое значение: $c = 64$

2.5 Нарушено требование 2: возвращаемое значение должно быть равно a^b

Исправление

В текущем цикле возводится в квадрат r , хотя при обработке битов справа налево нужно возводить в квадрат основание (или перейти на обработку битов слева направо). Ниже — исправление через обработку битов слева направо.

```
public int pow(int a, int b) {
    int r = 1;
    int bit = Integer.highestOneBit(b);

    while (bit != 0) {
        r *= r;
        if ((b & bit) != 0) r *= a;
        bit >>= 1;
    }

    return r;
}
```

Ошибка 3

При $b \leq 0$ метод `pow` класса `Power` возвращает 0 или некорректное значение вместо 1 (13:50 13.02.26)

Контекст

2.1 До вызова: `int a=5, int b=0`, создан `Power power`

2.2 Воздействие: вызов `power.pow(a,b)`

2.3 Фактическое значение (в исходной реализации): 0

2.4 Ожидаемое значение: 1

2.5 Нарушены требования 2.a и 2.b: при $b == 0$ и при $b < 0$ должен возвращаться 1

Исправление

Добавлена явная обработка $b \leq 0$.

```
public int pow(int a, int b) {
    if (b <= 0) return 1;

    int r = 1;
    int bit = Integer.highestOneBit(b);

    while (bit != 0) {
```

```
    r *= r;
    if ((b & bit) != 0) r *= a;
    bit >>= 1;
}

return r;
}
```

Ошибка 4

При переполнении метод pow класса Power возвращает результат по правилам переполнения int (mod 2^{32}), а требуется mod 2^{31} (14:15 13.02.26)

Контекст

2.1 До вызова: `int a=3, int b=20`, создан `Power power`

2.2 Воздействие: вызов `power.pow(a,b)`

2.3 Фактически возвращаемое значение (после исправлений 1–3): отрицательное число из-за переполнения `int` при умножениях

2.4 Ожидаемое значение: `1339300753`, так как $3^{20} = 3486784401$, а $3486784401 \bmod 2^{31} = 1339300753$

2.5 Нарушено требование 2.c: при переполнении возвращать результат по модулю 2^{31}

Исправление

Умножения переведены в `long`, после каждого умножения оставляются младшие 31 бит (эквивалент mod 2^{31}). Знак для отрицательного `a` при нечетной степени возвращается отдельно. Также учтен случай `a == Integer.MIN_VALUE`, чтобы не ломаться на `Math.abs`.

```
public int pow(int a, int b) {
    if (b <= 0) return 1;

    long mask = 0x7fffffffL; // 2^31 - 1
    boolean neg = a < 0 && (b & 1) != 0;

    long x;
    if (a == Integer.MIN_VALUE) x = 1L << 31;
    else x = Math.abs(a);
    x &= mask;

    long r = 1;
    int bit = Integer.highestOneBit(b);

    while (bit != 0) {
        r = r * r & mask;
        if ((b & bit) != 0) r = r * x & mask;
        bit >>= 1;
    }

    if (neg) r = -r;
    return (int) r;
}
```

```
}  
  
int res = (int) r;  
if (neg && res != 0) res = -res;  
return res;  
}
```